Hierarchical Structured Neural Network: Efficient Retrieval Scaling for Large Scale Recommendation

Kaushik Rangadurai
krangadu@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Siyang Yuan
syyuan@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Minhui Huang
mhhuang@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Yiqun Liu
yiqliu@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Golnaz Ghasemiesfeh
golnazghasemi@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Yunchen Pu
pyc40@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Haiyu Lu
hylu@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Xingfeng He
xingfenghe@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Fangzhou Xu
fxu@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Andrew Cui
andycui97@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Vidhoon Viswanathan
vidhoon@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Lin Yang
ylin1@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Liang Wang
liangwang@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Jiyan Yang
chocjy@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

Chonglin Sun
clsun@meta.com
Meta Platforms Inc.
Sunnyvale, CA, USA

ABSTRACT

Retrieval, the initial stage of a recommendation system, is tasked with down-selecting items from a pool of tens of millions of candidates to a few thousands. Embedding Based Retrieval (EBR) has been a typical choice for this problem, addressing the computational demands of deep neural networks across vast item corpora. EBR utilizes Two Tower or Siamese Networks to learn representations for users and items, and employ Approximate Nearest Neighbor (ANN) search to efficiently retrieve relevant items. Despite its popularity in industry, EBR faces limitations. The Two Tower architecture, relying on a single dot product interaction, struggles to capture complex data distributions due to limited capability in learning expressive interactions between users and items. Additionally, ANN index building and representation learning for user and item are often separate, leading to inconsistencies exacerbated by representation (e.g. continuous online training) and item drift (e.g. items expired and new items added). In this paper, we introduce the Hierarchical Structured Neural Network (HSNN), an efficient deep neural network model to learn intricate user and item interactions beyond the commonly used dot product in retrieval tasks, achieving sublinear computational costs relative to corpus size. A Modular Neural Network (MoNN) is designed to maintain high expressiveness for interaction learning while ensuring efficiency. A mixture of MoNNs operate on a hierarchical item index to achieve extensive computation sharing, enabling it to scale up to large corpus size. MoNN and the hierarchical index are jointly learnt to continuously adapt to distribution shifts in both user interests and item distributions. HSNN achieves substantial improvement in offline evaluation compared to prevailing methods. HSNN has been successfully deployed in Meta's ads recommendation system with significant online metric gains, demonstrating the effectiveness of the proposed approach in the production environment.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

CCS CONCEPTS

• **Computing methodologies** → **Machine Learning**.

KEYWORDS

Deep Retrieval, Clustering, Recommendation Systems

ACM Reference Format:
Kaushik Rangadurai, Siyang Yuan, Minhui Huang, Yiqun Liu, Golnaz Ghasemiesfeh, Yunchen Pu, Haiyu Lu, Xingfeng He, Fangzhou Xu, Andrew Cui, Vidhoon Viswanathan, Lin Yang, Liang Wang, Jiyan Yang, and Chonglin Sun. 2024. Hierarchical Structured Neural Network: Efficient Retrieval Scaling

for Large Scale Recommendation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Machine learning plays a crucial role in recommendation systems in identifying potential user interests. To tackle the vast number of candidates per request, industry practices [6, 10] often employ a cascade of recommendation systems with increasing computational cost. The first stage, known as the retrieval stage, narrows down millions of candidates to a few thousand.

The Retrieval stage has strict infrastructure constraints, making model architecture like Siamese Networks [2] a common choice. Siamese networks, or also called Two Tower, incorporate a late fusion technique where each tower outputs a fixed-sized representation. User tower uses user-only features (e.g. user country or user click history) and the user tower is computed at query-time. Item (ad) tower uses item-only features (e.g. content or topic of item) and item tower is computed asynchronously which is usually triggered by item updates. A single dot-product interaction between the user and item embeddings produces a logit for predicting likelihood of engagement.

Despite its popularity, the Two Tower model architecture has limitations due to its late fusion technique. This constrains the learning of sophisticated interaction between user and item, forcing features to belong to either the user (e.g. user engaged videos) or item (e.g. content of item) and prohibiting early interactions through <user, item> interactions features (e.g. user's historical engagement of topics of this item) or model based interactions like NeuMF [16], DCN [38] and DHEN [44] that are common in the ranking stages.

After training the Two Tower model, the user and item towers are deployed independently. Item embeddings are computed asynchronously and used to create a vector index using systems like Manas ([8, 9]) and FAISS ([15, 18]). Embedding Based Retrieval (EBR) [20] with approximate nearest neighbors (ANN) search [21] algorithms are commonly used to retrieve top relevant candidates. Despite their popularity in the industry [8, 9, 15, 18], there is inconsistency in the item embedding from index building time to scoring time. There are two main factors contributing to the dynamic nature of the item embedding distribution: **a)** The model is continuous learning and weight updates through online training, which leads to an evolving embedding distribution and **b)** The item distribution itself is also changing over time, with expired items being removed and new items being added to the index.

NeuMF [16], DCN [38] and DHEN [44] are advanced architectures, but their computational complexity makes them unsuitable for direct adoption in retrieval. Previous works such as Tree-based Deep Model (TDM) [46] and Deep Retrieval (DR) [11] have proposed methods to enable advanced neural networks beyond two tower models in retrieval. However, these methods are limited by using the same model architecture across the tree hierarchy, which constrains further scaling up of model complexity. Moreover, the architectures proposed in TDM and DR lack expressiveness in capturing user and item interactions, failing to leverage <user, item> interaction features. Furthermore, they require iterative training to learn the hierarchy with methods such as Expectation–Maximization (EM), which can be challenging and inefficient to optimize.

We propose Hierarchical Structured Neural Network (HSNN) that can be used as a drop-in replacement to any Embedding Based Retrieval (EBR) system. HSNN brings deep neural networks to retrieval with the capability of searching through the entire corpus with both accuracy and efficiency improvements. The primary contributions of the paper include:

- We introduce HSNN which is capable of handling tens to hundreds of million items with sublinear inference cost. Concretely: **a)** Modular Neural Network (MoNN) is established to efficiently learn complex interactions among users and items with deep neural networks, **b)** A highly optimized feature transformation algorithm is introduced in MoNN to consume <user, item> interaction features. **c)** Multiple MoNNs with different complexity operate in harmony on a hierarchical index, where items falling under the same index node could share the majority of computations.
- We propose a gradient descent based algorithm to jointly learn the advanced neural network and hierarchical index thereby eliminating inconsistencies in neural network learning and index learning due to online training and item drift.
- We demonstrate the effectiveness of our proposed approach and showcase substantial performance improvements in both offline evaluation and online A/B experiments.

2 MODULAR NEURAL NETWORK (MONN)

In this section, we introduce a new model architecture called Modular Neural Network (MoNN) and show MoNN is a more powerful model architecture leading to better performance with flexibly of operating under different infrastructure constraints.

The Modular Neural Network (MoNN) enhances the learning of sophisticated user and item interactions beyond a single dot product while maintaining high efficiency. As shown in Figure 1, it achieves this through a modularized design comprising separate modules for user representations (**User Tower**), item representations (**Item Tower**) and the interaction among user and item (**Interaction Tower**). MoNN offers high flexibility, allowing for control over the complexity of each tower to balance expressiveness and computational cost effectively.

User Tower. The User Tower processes user features to generate a fixed-size user embedding. These features can be dense (e.g., number of clicks by the user) or sparse (e.g., user engaged videos). Sparse features are input into an embedding table, and all feature embeddings are concatenated and fed into a Tower Network. The user tower outputs an embedding of size $\text{num\_embed}_{user} * \text{dim}_{user}$ and is computed at query time. Given the user tower only needs to be computed once and shared across a vast number of items, it could scale up to very high complexity.

Item Tower. The Item Tower mirrors the User Tower, processing item dense (e.g. item historical click through rate) and sparse features (e.g. content of item). Sparse features are input into an embedding table, and all feature outputs are concatenated and fed into a Tower Network, producing an embedding of size $\text{num\_embed}_{item} * \text{dim}_{item}$.

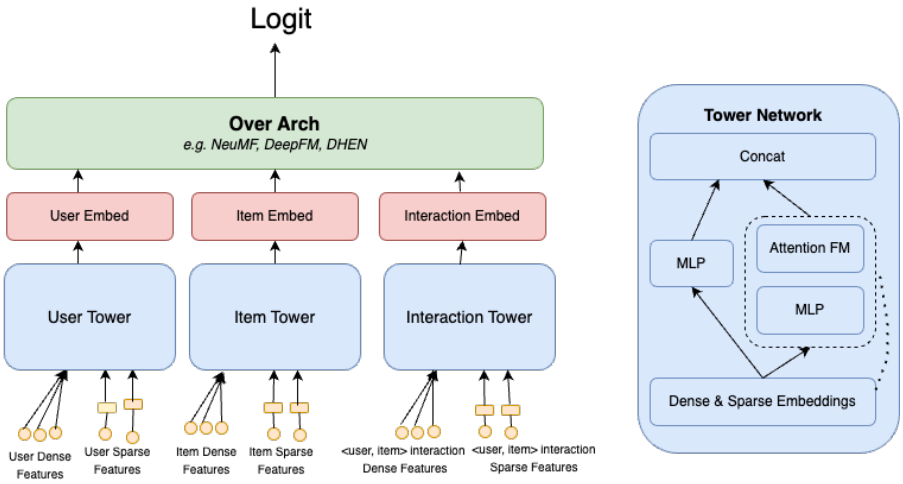

Figure 1: MoNN architecture design.

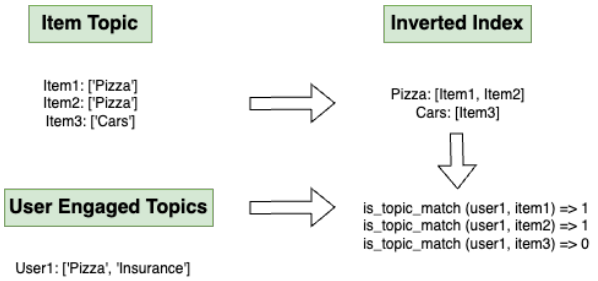

Figure 2: Inverted Index Based Interaction Features (I2IF)

Interaction Tower. The Interaction Tower uses <user, item> interaction features (dense and sparse) as input. It follows a similar architecture as user tower and item tower to produce an embedding of dimension num_embed$_{interaction}$ * dim$_{interaction}$. The Interaction Tower is computationally intensive as it runs for each pair of user and item. To minimize the computation cost for <user, item> interaction features, Inverted Index Based Interaction Features (I2IF) is introduced where an inverted index is employed for indexing item information with user information written as a query to perform efficient crossing computation. In the example shown in Figure 2, the item category feature (for each item) is kept in an inverted index, and the user feature (item categories engaged by the user) is passed through the query to produce a dense interaction feature. I2IF could be applied for sparse interaction feature generation in a similar manner.

OverArch. Sitting atop the three underlying towers (User Tower, Item Tower and Interaction Tower), the OverArch component takes their outputs as input. It employs a DHEN style [44] model architecture to generate logits. Notably, unlike traditional Two Tower models, dim$_{item}$, dim$_{user}$ and dim$_{interaction}$ could be different and num_embed$_{user}$, num_embed$_{interaction}$ and num_embed$_{item}$ could be greater than one.

Tower Network. The neural networks for User Tower, Item Tower and Interaction Tower follow a stacked architecture similar to EDCN [5]. It includes an MLP layer to capture implicit feature interactions and an AttentionFM [40] layer for explicit pairwise feature interactions.

Training Setup. The MoNN model is trained on a large training dataset with clicks and conversions as labels, impressions (non click or conversion) as negatives and additional unlabeled data used for semi-supervised learning to debias the model. A wide range of features (O(1000)) are used as input to the model. The model is optimized for multiple tasks - for e.g. click task and conversion task. The model is then trained using a multi-task cross-entropy loss:

$$L_{sup} = \frac{-1}{S}\sum_{i=1}^{S}\sum_{t=1}^{T} w_t(y_{ti}log(\hat{y}_{ti}) + (1-y_{ti})(log(1-\hat{y}_{ti}))) \quad (1)$$

$$L_{unsup} = \frac{-1}{S}\sum_{i=1}^{S}\sum_{t=1}^{T} distil(\hat{y}_{ti}, y_{ti}^{model}) \quad (2)$$

$$L = L_{sup} + L_{unsup} \quad (3)$$

where w_t is the weight for task t, $t = 1, 2, ...T$, representing its importance in the final loss. $y_{ti} \in 0, 1$ is the label for sample i in task t. $\hat{y}_{ti}$ is the predicted value of the model for sample i in task t and y_{ti}^{model} is the soft label generated by the MoNN model or another stronger model. S is the number of samples.

Online Training. The retrieval model is trained through online learning where the data is ingested in a continuous stream without requiring materialization and a new model snapshot is created periodically (e.g. every few minutes) for online serving. This online training and frequent snapshot publishing, allows MoNN to consistently provide up-to-date predictions and catch-up with the item drift in the ecosystem.

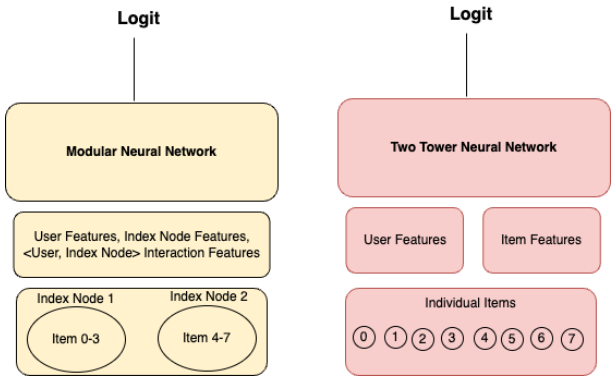

Figure 3: Neural Networks operating at different item granularities (index node vs item)

3 HIERARCHICAL STRUCTURED NEURAL NETWORK (HSNN)

While MoNN demonstrates promising ability to capture complex user and item interactions, its potential is hindered by the sheer scale of items. To overcome this limitation, we introduce HSNN, which scales up MoNN with sublinear cost in terms of corpus size. The following section delves into the details of how HSNN achieves this efficient scaling.

Granularity of Entity Representation Matters. A fundamental challenge in deploying state-of-the-art neural networks in the Retrieval stage is the vast cardinality of items. With thousands or tens of thousands times more items than deep neural neural networks in recommendation typically handle, this leads to:

- Prohibitive computational costs for advanced neural networks.
- Exorbitant I/O access and computation costs for utilizing large amounts of features, particularly cross <user, item> features.

However, reducing the cardinality of items by K times (e.g., building an item index with each index node containing K items) could theoretically enable the use of K time complex models in the Retrieval without increasing serving costs.

Motivating Example. Consider an item index with C index nodes, each containing K items. By assuming items within an index node share similar topics, we can reduce the complexity of candidate selection from O(K*C) to O(K+T), where T is the number of items to return. This allows for adopting more advanced neural networks (**Model 1**) to learn sophisticated interactions between user and item (at index granularity). In contrast, Two Tower models (**Model 2**) are limited to learning basic interactions through a single dot product without any <user, item> interaction features. The comparison of Model 1 and Model 2 are shown in Figure 3.

While both models have pros and cons, **Model 2** captures finer-grained signals at the item level, whereas **Model 1** leverages advanced neural networks with larger signal volumes at the item index level. Ideally, a retrieval model paradigm should combine the strengths of both approaches.

HSNN Introduction. To harness the strengths of both models in Figure 3, we introduce HSNN to learn a mixture of multiple models on top of a hierarchical item index with multiple different granularities (i.e. different granularity of item representations). By carefully designing the granularity of item index (i.e. how many items share an index node), HSNN enables the deployment of advanced ML architectures in Retrieval, capitalizing on computation sharing across entities. The coarser granularity an index is, the more computation sharing is achieved. With a hierarchical item index, HSNN can adopt a mixture of ML models with varying complexities to jointly optimize for personalization power.

HSNN consists of two primary components: (1) the hierarchical index and (2) the neural network design for each layer of the index hierarchy. In Section 3.1, we detail the model architecture design, while Section 3.2 delves into the learning of the hierarchical index.

3.1 Model Architecture

HSNN is a mixture of Modular Neural Networks (MoNNs) operating on top of the item hierarchy. While HSNN can be applied to N layers of hierarchy, Figure 4 shows a 3-layer HSNN model architecture. In this architecture, HSNN operates on a hierarchical index with three layers: L_1 index, L_2 index, and L_3 index. The L_1 index operates on the most coarse index granularity and it is able to take advantage of a MoNN model architecture with highest complexity (through computation sharing) and a wide range of interaction features (<user, L_1 index node>). On the other hand, the L_N index (where N=3 in this diagram) operates on the item granularity and leverages a MoNN model architecture with lowest complexity and a minimal set of <user, item> interaction features. The three MoNN modules are combined using an ensemble layer. This approach allows final prediction to leverage multiple MoNNs with different model complexity to consume different granularity of features resulting in a more accurate prediction.

Features. MoNN Small processes features at the individual item level, utilizing user features, item features and <user, item> interaction features. In contrast, MoNN Medium and MoNN Large operate at a coarser granularity (L_2 index and L_1 index respectively) and consume user features, index node features and <user, index node> interaction features where index node is replaced by a representative item for feature computation. More information on representative item selection is provided in Algorithm 1.

Loss Function. There are N supervision losses, 1 for each layer in the hierarchy with 1 <user, item> supervision loss and (N-1) <user, index node> supervision loss. Each of these is also calibrated to ensure that the model doesn't under-predict or over-predict. As discussed in the MoNN section, multi-task cross-entropy loss is used to optimize the model.

$$L_{sup} = \frac{-1}{S}\sum_{i=1}^{S}\sum_{j=1}^{N}\sum_{t=1}^{T} w_t(y_{ti}log(\hat{y}_{ti}^{L_j}) + (1-y_{ti})(log(1-\hat{y}_{ti}^{L_j}))) \tag{4}$$

where w_t is the weight for task t, $t = 1, 2, ...T$, representing its importance in the final loss. $y_{ti} \in 0, 1$ is the label for sample i in task t. $\hat{y}_{ti}^{L_j}$ is the predicted value of the model for sample i in task

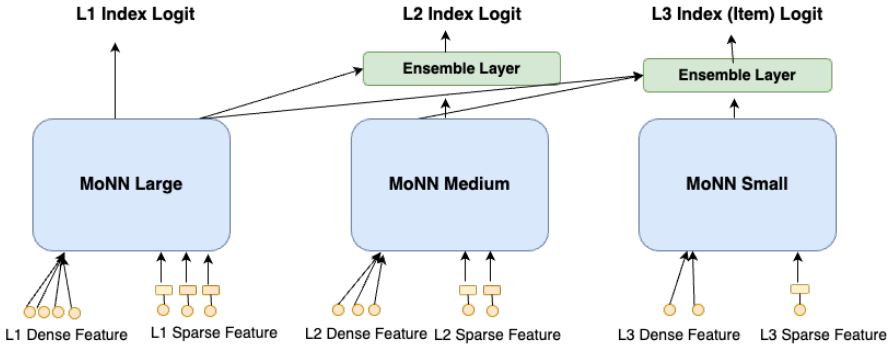

Figure 4: The model framework for 3-layer HSNN with 3 MoNNs (MoNN Large, MoNN Medium, and and MoNN Small) operating on a 3-layer hierarchical index (L_1 index, L_2 index and L_3 index). L_1 and L_2 has coarse granularity of item representations, while L_3 layer preserving full item granularity. Each index uses sparse and dense features belonging to 3 types: user, index node and <user, index node> interaction features.

t at layer L_j. S is the number of samples and N is the number of layers in HSNN.

3.2 Learning Hierarchical Index

The hierarchical index can be constructed using three approaches:

(a) **Exploiting existing data structures**: Leveraging the natural hierarchy of Item -> Item Creator -> Item Creator Group in the existing data structure.
(b) **Clustering item embeddings**: Applying traditional clustering methods (e.g., k-means) to obtain item-index assignments based on item embeddings.
(c) **Joint optimization**: Co-optimizing index assignment and the MoNN Item module.

While approaches **a)** and **b)** have proven effective by independently learning the hierarchical index and retrieval model, we hypothesize approach **c)** can yield additional gains by co-training individual components. Therefore, we propose a Jointly Optimized indexing and MoNN **(JOIM)**, which can produce better results and generalize to learning sophisticated interactions between user and item. In the following section, we introduce a gradient descent based algorithm that learns both index assignment and representation, alongside MoNN model training. This would cover Learning to Index (LTI) for learning single layer index, learning a hierarchy of index through residual learning and the joint learning of hierarchical index and MONN.

3.2.1 Learning To Index (LTI). We propose a novel Learning-to-Index (LTI) algorithm that offers three key advantages:

- **Seamless integration**: Our gradient-descent-based approach can be easily incorporated into existing training infrastructure.
- **Lightweight and versatile**: The algorithm is designed to be computationally efficient, allowing for integration with various model architectures without significant impact on training QPS.
- **Gradient-friendly**: By overcoming the argmax operator in index assignment, our algorithm enables gradients to flow through, facilitating more effective optimization.

Algorithm. The LTI algorithm takes the item embedding from a MoNN model as input and learns coarse index node embeddings for the item by minimizing the L2 distance between them. To address the argmax operator (which identifies the closest index node to an item), it employs an attention-based method by taking the item embedding as query, learnable embeddings for index nodes as keys and values and L2 distance based attention to calculate the index embedding for them. The LTI algorithm is described in detail in Algorithm 1. This approach calculates a soft assignment of the item to each index node based on a normalized L2 distance (line 6). It uses this to compute the item representation in the index by taking a weighted sum of index node embeddings (line 7). A key advantage of the soft assignment learning is that it allows items to belong to multiple index nodes with varying degrees of membership thereby capturing the uncertainty in the data.

Algorithm 1 Learning To Index (LTI)

Input: MoNN model, num nodes per index (K)
Output: $mapping_{item,index}$ m and $representative\_tem_{index}$ r

1: Initialize embeddings$_{index}$ $\{c_k\}_{k=1}^{K}$
2: **for** each j in the batch **do**
3: Compute MoNN user embedding u_j and item embedding v_j
4: $distance_{item,index}$ d(j,k) = $\|v_j - c_k\|^2$
5: $mapping_{item,index}$ $m_j = \text{argmin}_k\, d(j,k)$
6: $affinity_{item,index}$ $a_k = \frac{e^{-\alpha * d(j,k)}}{\sum_{k'} e^{-\alpha * d(j,k')}}$
7: $embedding_{item,index}$ $\bar{c} = \sum_k a_k c_k$
8: add supervision $logloss_{index}(y, \langle u_j, \bar{c} \rangle)$
9: **end for**
10: Initialize representative_$item_{index}$
11: **for** each item j in corpora V **do**
12: **for** each k in K **do**
13: representative_$item_{index,k}$ r_k = j if $d(j,k)$ is closer than existing value
14: **end for**
15: **end for**
16: Publish $mapping_{item,index}$ m and representative_$item_{index}$ r

3.2.2 Residual Learning. Inspired by Residual Quantized Variational AutoEncoder (RQ-VAE) [24] [43], the residue between input item embedding and the corresponding index node embedding in one index layer is passed as input to the next index layer.

Concretely, given the item embedding v, the N level residual learning algorithm initialized the index nodes $\{c_n^k\}_{k=1,...,K;n=1,...,N}$ and recursively quantizes the residual vector r_n at each level n.

$$r_1 = v \tag{5}$$

$$r_n = r_{n-1} - \bar{c}_{n-1} \tag{6}$$

where $\bar{c}_{n-1}$ is the $\text{embedding}_{item,index}$ (line 7 in Equation 1). In each level n, the quantized item embedding is computed as $q_n = \sum_{t=1}^{n-1} \bar{c}_t$. The reconstruction loss is computed as:

$$reconstruction\ loss = \|q_N - v\|^2 \tag{7}$$

The magnitude of residuals decreases when further moving down the hierarchy. As a result, a more coarse index layer identifier expresses more general concepts, while a fine-grained index layer captures more detailed notions.

3.2.3 Joint Optimization of Index and MoNN (JOIM). As LTI is a gradient descent approach for learning hierarchical index, it could naturally be jointly trained with MoNN. Following are some key considerations to enable it work:

- **Gradient Flow**: In separate index learning, item index assignment is solely determined by item embeddings. However, joint optimization allows index assignment to be influenced by <user, item> supervision. Gradients from this supervision flow through the item tower to the LTI module, impacting item node embeddings and index assignments.
- **Interaction Tower**: Figure 4 illustrates how the MoNN module processes interaction features at both L_1 and L_2 index granularities to derive the interaction embedding. Yet it is challenging to access index level interaction features during training. To address this, we introduce a dedicated interaction tower for each index layer, utilizing only user and item features as inputs. Additionally, an auxiliary mean-squared-error loss is applied between the index and item-level interaction towers to aid in convergence.

3.2.4 Training Optimization. A few additional techniques are introduced to improve the training stability including softmax temperature scheduler, balanced index distribution and warmup strategy.

Softmax Temperature Scheduler. HSNN uses features from the representative item (line 13 from Algorithm 1) during serving. However, during training the LTI algorithm uses a soft <item, index node> assignment. To mitigate the discrepancy, a scheduler is applied by gradually increasing the temperature (alpha), transitioning from soft assignment in the initial phase to a hard assignment later on. Small values of alpha yield a balanced distribution of the item-to-index assignment while large values of alpha result in a skewed distribution. The scheduler is based on the following function:

$$alpha = max\_alpha * \frac{current\_iter^{exp}}{max\_iters^{exp}} \tag{8}$$

Balanced Index Distribution. The index learning often suffers from cluster collapse, where the model utilizes only a limited subset of index nodes. A balanced index distribution is crucial to enable the use of neural network models with high complexity. We employ FLOPs regularizer to address this problem. The motivation stems from Paria et al. [31] which penalizes the model if all items are assigned to the same index node or if the distribution of <item, index node> assignment is imbalanced. Sparsity loss is introduced to minimize the sum of squares of the mean soft assignment. Given this is sensitive to smaller batches, data from most recent K batches is pooled and the FLOPs regularizer is applied on the pooled soft assignment matrix (K * batch_size, num_index_nodes).

Warmup. A linear warm up strategy is employed for the index loss weight to gradually increase the learning rate. This approach stabilizes the model parameters and mitigates the issue of item assignment oscillating between index nodes during the initial training phase.

4 OFFLINE ABLATION STUDIES

Ablation studies on HSNN are performed to confirm the value of key components. We use 2 offline metrics - Normalized Entropy (NE) and Recall@K to measure the performance.

Normalized Entropy (NE). Normalized Entropy [17] is equivalent to the average log loss per impression divided by what the average log loss per impression would be if a model predicted the average empirical CTR/CVR of the training data set. The lower the value, the better the model's predictions. Any number above **0.05%** NE gain is considered to be significant.

Recall@K. Recall@K gives a measure of how many of the relevant items are present in top K out of all the relevant items, where K is the number of recommendations generated for a user. A recall gain of **0.5%** is considered to be significant.

4.1 Advanced Model Architecture

This section explores whether MoNN can improve accuracy. An offline study was conducted, scoring all ads using MoNNs. We compared the Two Tower model with four MoNN variants (detailed in Section 2) as shown in Table 1. The complexity of the model is measured in Floating Point Operations (FLOPs), indicating the amount of computations needed for a single inference. All metrics reported (inference FLOPs, eval NE, infra cost and recall) are relative to the Two Tower XS model. The MoNN Small, Medium, Large, and XL models exhibit progressively larger orders of magnitude in FLOPs.

MoNN Small achieves 0.29% NE gain over the Two Tower model architecture, highlighting the benefits of sophisticated <user, item> interaction learning provided by the Interaction Tower, Over-Arch, and <user, item> interaction features in MoNN. While large-scale MoNN variants demonstrate significant NE gains, their exponential infra cost computation makes them impractical for the retrieval stage.

4.2 Hierarchical Structured Neural Networks (HSNN)

Table 1 demonstrates the outsized gains from HSNN (up to 1.46% NE gain and 10% recall lift) over Two Tower architecture with magnitude less infra cost compared to vanilla MoNN, producing high return-on-investments (ROI) in Meta Ads. The relative infra cost shown in the table measures the amount of hardware capacity

Model Architecture	FLOPs	Eval NE (↓)	Theoretical Cost	Infra Cost (↓)	Recall (↑)
Two Tower (XS)	0.25x	baseline	M_{XS} * V	1x	0
MoNN Small	1x	-0.29%	M_S * V	2.5x	+2.4%
MoNN Medium	50x	-0.70%	M_M * V	17.3x	+4.2%
MoNN Large	30,000x	-1.70%	M_L * V	24.6x	+9.4%
MoNN XL	200,000x	-2.20%	M_{XL} * V	64x	+12%
EBR (Two Tower)	0.25x	+0.03%	M_{XS} * I_1	0.49x	-0.1%
2-layer HSNN (L_1: MoNN Small, L_2: TTSN)	1.25x	-0.23%	M_S * I_1 + M_{XS} * V	1.7x	+2.2%
2-layer HSNN (L_1: MoNN Medium, L_2: MoNN-Small)	51x	-0.47%	M_M*I_1 + M_S * V	3.3x	+3.6%
2-layer HSNN (L_1: MoNN Large, L_2: MoNN Small)	30,001x	-0.97%	M_L*I_1 + M_S * V	3.9x	+6%
3-layer HSNN (L_1: MoNN XL, L_2: MoNN Large, L_3: MoNN Small)	230,0001x	-1.46%	M_{XL}*I_1 + M_L*I_2 + M_S * V	4.5x	+10%

Table 1: HSNN enables more complex model architectures like MoNN XL to the retrieval stage. Every scale-up brings in an order of magnitude gain. I1, I2, V are in the order of O(1,000), O(100,000), O(10,000,000) respectively. Any NE gain above 0.05% and recall above 0.5% is considered significant.

needed to serve the model in production with the same throughput and QPS. HSNN clearly distinguishes itself by effectively utilizing the hierarchical index with more complex MoNN architectures. In all of the HSNN experiments, the MoNN model is jointly optimized with the hierarchical index using the LTI algorithm.

For theoretical analysis, we present the cost of serving the HSNN model based on the following parameters: $\mathbf{I}_1$ (number of nodes in L_1 index), $\mathbf{I}_2$ (number of nodes in L_2 index) and **V** (number of items in the corpus), $\mathbf{M}_{XS}$ denotes the cost to serve Two Tower model, $\mathbf{M}_S$ denotes the cost to serve MoNN Small, $\mathbf{M}_M$ the cost to serve MoNN Medium, $\mathbf{M}_L$ the cost to serve MoNN Large and $\mathbf{M}_{XL}$ the cost to serve MoNN XL.

4.3 Joint Optimization

As shown in Table 2, the Joint Optimization (**JOIM**) brings in about 0.15% NE gain on the HSNN with separate index learning (**SIL**) with the same model architecture. Considering the large item corpora, k-means clustering is used for its good scalability. The LTI algorithm is also compared with the **EM** variation where the MoNN model architecture and the hierarchical index are jointly learnt in an alternative way i.e.train a MoNN model with the current hierarchical index until convergence and use the converged MoNN model to update the hierarchical index structure. And LTI demonstrated better performance compared to the EM variant. Note that the infra cost for JOIM is the same as HSNN. In all the experiments, the model complexity and number of nodes is kept constant for comparison purposes.

As shown in Table 3, an ablation study is performed on the various components of the LTI algorithm. The NE gains demonstrate the effectiveness of the training optimization strategies.

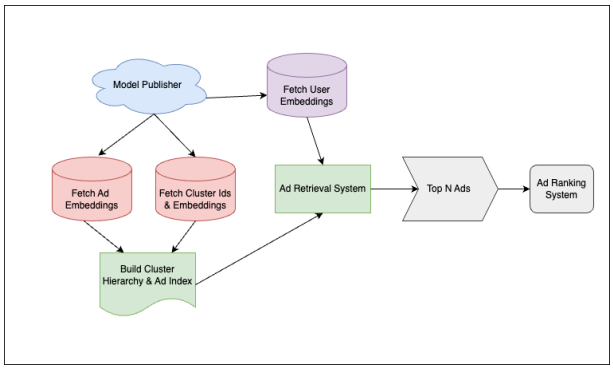

Figure 5: System architecture of the retrieval system.

Model Architecture	Eval NE (↓)
HSNN w/ **SIL**	baseline
HSNN + JOIM w/ **EM**	-0.11%
HSNN + JOIM w/ **LTI**	-0.15%

Table 2: Ablation study on HSNN (L_1: MoNN Small, L_2: Two Tower) shows that Joint Optimization brings in 0.15% NE gain.

Ablation Study	Eval NE (↓)
HSNN (MoNN Small)	baseline
- Softmax Temperature scheduler	+0.1%
- warmup strategy	+0.03%
- balanced index distribution	+0.05%

Table 3: Ablation study of various LTI components.

Model Architecture	Online Topline Metric (↑)
Two Tower	-0.21%
MoNN Small	baseline
HSNN + SIL (L_1: MoNN Medium, L_2: MoNN Small)	+1.06%
HSNN + JOIM (L_1: MoNN Medium, L_2: MoNN Small)	+1.22%
HSNN + JOIM (L_1: MoNN Large, L_2: MoNN Small)	+2.57%

Table 4: Online performance of HSNN in Meta Ads production. Any online topline metric gain above 0.1% is considered significant.

5 ONLINE EXPERIMENTS

Figure 5 illustrates the production deployment of the HSNN in Meta Ads. At query time, the user tower is executed to generate user embeddings, while the item tower operates asynchronously to index item embeddings. The <user, item> interaction features are generated on the fly using the I2IF framework, feeding into the interaction tower to produce interaction embeddings in real-time. All embeddings are then passed to the OverArch and Ensemble layer to compute the logit at query time.

The HSNN framework has been widely deployed to the Meta Ads Retrieval system for over 2 years. Prior to the deployment of HSNN, we've launched MoNN Small architecture to production given the relatively low infra cost. As shown in Table 4, online A/B tests demonstrate that HSNN led to **2.57%** online ads metric gains (0.1% gain is considered as statistically significant).

6 RELATED WORK

Embedding Based Retrieval. Embedding based retrieval (EBR) has been successfully applied in retrieval for search and recommendation systems [25, 32]. [20] expands this concept by integrating text, user, context, and social graph information into a unified embedding, enabling the retrieval of documents that are both relevant to the query and personalized for the user. On the systems side, many have developed or deployed approximate nearest neighbor (ANN) algorithms that can identify the top-k candidates for a given query embedding in sub-linear time [8, 9, 18]. Efficient Maximum Inner Product Search (MIPS) or ANN algorithms include tree-based methods [19, 30], locality sensitive hashing (LSH) [33, 34], product quantization (PQ) [13, 22], hierarchical navigable small world graphs (HNSW) [27], etc. Another research direction focuses on encoding vectors in a discrete latent space. Vector Quantized-Variational AutoEncoder (VQ-VAE) [37] proposes a simple yet powerful generative model to learn a discrete representation. HashRec [23] and Merged-Averaged Classifiers via Hashing [29] employ multi-index hash functions to encode items in large recommendation systems. Hierarchical quantization methods like Residual Quantized Variational AutoEncoder (RQ-VAE) [43] and Tree-VAE [28] are also used to learn tree structure of a vector. However, these systems often assume a stable item vocabulary and do not account for embedding distribution shifts.

Clustering Methods for ANN. Clustering algorithms can be broadly categorized into hierarchical and partitional methods. Agglomerative clustering [41], a hierarchical approach, begins with numerous small clusters and progressively merges them. Among partitioning techniques, K-means [26] is the most well-known, aiming to minimize the sum of squared distances between data points and their nearest cluster centers. Other approaches in this category include expectation maximization (EM) [7], spectral clustering [14], and non-negative matrix factorization (NMF) [3] based clustering.

Advanced Model Architectures. Model architectures such as DCN, DHEN promote early feature interactions within the model. Due to their higher serving costs, these architectures are typically utilized in the ranking stages with a limited number of candidates to evaluate. Recently, generative retrieval has emerged as a novel paradigm for document retrieval [1, 4, 35, 36, 39]. This approach parallels our work, treating items as documents and users as queries. The learned codebook in generative retrieval is akin to the item nodes in the hierarchical index proposed in this paper.

Joint Optimization. The line of work that most resembles our work are the joint optimized systems where the item (ad) hierarchy and the large-scale retrieval model are jointly optimized. Gao et al. [12] propose Deep Retrieval (DR), which encodes all items in a discrete latent space with an end-to-end neural network design. Deep Retrieval has constraints in terms of model architecture and features that can be utilized. Zhu et al. [46] propose a novel tree-based method which can provide logarithmic complexity w.r.t. corpus size even with more expressive models such as deep neural networks. In this research area, tree-based methods [42, 45, 47] are research. These approaches map each item to a leaf node within a tree structure and jointly learn an objective function for both the tree structure and model parameters.

7 CONCLUSION AND NEXT STEPS

In this paper we presented a deployed retrieval model called Hierarchical Structured Neural Network (HSNN). HSNN is a powerful framework that can enable advanced neural networks to learn complex interaction among user and item features with high accuracy and efficiency. At Meta, HSNN has been successfully deployed to one of the largest Ads recommendation systems.

While hierarchical index has been heavily explored for items, it remains an important topic on how to pursue a similar strategy for users to further improve its performance. Additionally, as generative retrieval emerges, it'd be a promising direction to incorporate generative components into HSNN to further enhance its performance.

8 ACKNOWLEDGEMENTS

The authors would like to thank Le Fang, Tushar Tiwari, Wei Lu, Jason Liu, Trevor Waite, Shu Yan, Alexander Petrov, Dheevatsa Mudigere, Benny Chen, GP Musumeci, Yiping Han, Bo Long, Wenlin Chen, Santanu Kolay and others who contributed, supported and collaborated with us.

REFERENCES

[1] Michele Bevilacqua, Giuseppe Ottaviano, Patrick Lewis, Wen tau Yih, Sebastian Riedel, and Fabio Petroni. 2022. Autoregressive Search Engines: Generating Substrings as Document Identifiers. arXiv:cs.CL/2204.10628

[2] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. 1993. Signature Verification Using a "Siamese" Time Delay Neural Network. In *Proceedings of the 6th International Conference on Neural Information Processing Systems* (Denver, Colorado) *(NIPS'93)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 737–744.

[3] Deng Cai, Xiaofei He, Xuanhui Wang, Hujun Bao, and Jiawei Han. 2009. Locality Preserving Nonnegative Matrix Factorization. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence* (Pasadena, California, USA) *(IJCAI'09)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1010–1015.

[4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2021. Autoregressive Entity Retrieval. arXiv:cs.CL/2010.00904

[5] Bo Chen, Yichao Wang, Zhirong Liu, Ruiming Tang, Wei Guo, Hongkun Zheng, Weiwei Yao, Muyu Zhang, and Xiuqiang He. 2021. Enhancing Explicit and Implicit Feature Interactions via Information Sharing for Parallel Deep CTR Models. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management* (Virtual Event, Queensland, Australia) *(CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 3757–3766. https://doi.org/10.1145/3459637.3481915

[6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (Boston, Massachusetts, USA) *(RecSys '16)*. Association for Computing Machinery, New York, NY, USA, 191–198. https://doi.org/10.1145/2959100.2959190

[7] A. P. Dempster, N. M. Laird, and D. B. Rubin. 2018. Maximum Likelihood from Incomplete Data Via the EM Algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)* 39, 1 (12 2018), 1–22. https://doi.org/10.1111/j.2517-6161.1977.tb01600.x arXiv:https://academic.oup.com/jrsssb/article-pdf/39/1/1/49117094/jrsssb_39_1_1.pdf

[8] Ishita Doshi, Dhritiman Das, Ashish Bhutani, Rajeev Kumar, Rushi Bhatt, and Niranjan Balasubramanian. 2020. LANNS: A Web-Scale Approximate Nearest Neighbor Lookup System. arXiv:cs.IR/2010.09426 https://arxiv.org/abs/2010.09426

[9] Pinterest Engineering. 2021. *Manas HNSW Realtime: Powering Realtime Embedding-Based Retrieval.* Pinterest. https://medium.com/pinterest-engineering/manas-hnsw-realtime-powering-realtime-embedding-based-retrieval-dc71dfd6afdd

[10] Luke Gallagher, Ruey-Cheng Chen, Roi Blanco, and J. Shane Culpepper. 2019. Joint Optimization of Cascade Ranking Models. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining* (Melbourne VIC, Australia) *(WSDM '19)*. Association for Computing Machinery, New York, NY, USA, 15–23. https://doi.org/10.1145/3289600.3290986

[11] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2021. Deep Retrieval: Learning A Retrievable Structure for Large-Scale Recommendations. arXiv:cs.IR/2007.07203 https://arxiv.org/abs/2007.07203

[12] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2021. Deep Retrieval: Learning A Retrievable Structure for Large-Scale Recommendations. arXiv:cs.IR/2007.07203

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. https://doi.org/10.1109/TPAMI.2013.240

[14] Cuimei Guo, Sheng Zheng, Yaocheng Xie, and Wei Hao. 2012. A survey on spectral clustering. In *World Automation Congress 2012*. 53–56.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. arXiv:cs.LG/1908.10396 https://arxiv.org/abs/1908.10396

[16] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. arXiv:cs.IR/1708.05031 https://arxiv.org/abs/1708.05031

[17] Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, and Joaquin Quiñonero Candela. 2014. Practical Lessons from Predicting Clicks on Ads at Facebook. In *Proceedings of the Eighth International Workshop on Data Mining for Online Advertising* (New York, NY, USA) *(ADKDD'14)*. Association for Computing Machinery, New York, NY, USA, 1–9. https://doi.org/10.1145/2648584.2648589

[18] Jeff Johnson Hervé Jegou, Matthijs Douze. 2017. *Faiss: A library for efficient similarity search.* Facebook. https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/

[19] Michael E. Houle and Michael Nett. 2015. Rank-Based Similarity Search: Reducing the Dimensional Dependence. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37, 1 (2015), 136–150. https://doi.org/10.1109/TPAMI.2014.2343223

[20] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. https://doi.org/10.1145/3394486.3403305

[21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-scale similarity search with GPUs. arXiv:cs.CV/1702.08734

[22] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[23] Wang-Cheng Kang and Julian McAuley. 2019. Candidate Generation with Binary Codes for Large-Scale Top-N Recommendation. arXiv:cs.IR/1909.05475

[24] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive Image Generation using Residual Quantization. arXiv:cs.CV/2203.01941

[25] Yiqun Liu, Kaushik Rangadurai, Yunzhong He, Siddarth Malreddy, Xunlong Gui, Xiaoyi Liu, and Fedor Borisyuk. 2021. Que2Search: Fast and Accurate Query and Document Understanding for Search at Facebook. , 9 pages. https://doi.org/10.1145/3447548.3467127

[26] James MacQueen et al. 1967. Some methods for classification and analysis of multivariate observations. , 281–297 pages.

[27] Yu. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:cs.DS/1603.09320

[28] Laura Manduchi, Moritz Vandenhirtz, Alain Ryser, and Julia Vogt. 2023. Tree Variational Autoencoders. arXiv:cs.LG/2306.08984

[29] Tharun Medini, Qixuan Huang, Yiqiu Wang, Vijai Mohan, and Anshumali Shrivastava. 2019. Extreme Classification in Log Memory using Count-Min Sketch: A Case Study of Amazon Search with 50M Products. arXiv:cs.LG/1910.13830

[30] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240. https://doi.org/10.1109/TPAMI.2014.2321376

[31] Biswajit Paria, Chih-Kuan Yeh, Ian E. H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. arXiv:cs.LG/2004.05665

[32] Kaushik Rangadurai, Yiqun Liu, Siddarth Malreddy, Xiaoyi Liu, Piyush Maheshwari, Vishwanath Sangale, and Fedor Borisyuk. 2022. NxtPost: User to Post Recommendations in Facebook Groups. arXiv:cs.LG/2202.03645 https://arxiv.org/abs/2202.03645

[33] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). arXiv:stat.ML/1405.5869

[34] Ryan Spring and Anshumali Shrivastava. 2017. A New Unbiased and Efficient Class of LSH-Based Samplers and Estimators for Partition Function Computation in Log-Linear Models. arXiv:stat.ML/1703.05160

[35] Weiwei Sun, Lingyong Yan, Zheng Chen, Shuaiqiang Wang, Haichao Zhu, Pengjie Ren, Zhumin Chen, Dawei Yin, Maarten de Rijke, and Zhaochun Ren. 2023. Learning to Tokenize for Generative Retrieval. arXiv:cs.IR/2304.04171

[36] Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. 2022. Transformer Memory as a Differentiable Search Index. arXiv:cs.CL/2202.06991

[37] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2018. Neural Discrete Representation Learning. arXiv:cs.LG/1711.00937

[38] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. arXiv:cs.LG/1708.05123 https://arxiv.org/abs/1708.05123

[39] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Hao Sun, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, Xing Xie, Hao Allen Sun, Weiwei Deng, Qi Zhang, and Mao Yang. 2023. A Neural Corpus Indexer for Document Retrieval. arXiv:cs.IR/2206.02743

[40] Jun Xiao, Hao Ye, Xiangnan He, Hanwang Zhang, Fei Wu, and Tat-Seng Chua. 2017. Attentional Factorization Machines: Learning the Weight of Feature Interactions via Attention Networks. arXiv:cs.LG/1708.04617 https://arxiv.org/abs/1708.04617

[41] Jianwei Yang, Devi Parikh, and Dhruv Batra. 2016. Joint Unsupervised Learning of Deep Representations and Image Clusters. arXiv:cs.CV/1604.03628 https://arxiv.org/abs/1604.03628

[42] Ronghui You, Zihan Zhang, Ziye Wang, Suyang Dai, Hiroshi Mamitsuka, and Shanfeng Zhu. 2019. AttentionXML: Label Tree-based Attention-Aware Deep Model for High-Performance Extreme Multi-Label Text Classification. arXiv:cs.CL/1811.01727

[43] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. SoundStream: An End-to-End Neural Audio Codec. arXiv:cs.SD/2107.03312

[44] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, Yang Liu, Huayu Li, Yasmine Badr, Jongsoo Park, Jiyan Yang, Dheevatsa Mudigere, and Ellie Wen. 2022. DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction. arXiv:cs.IR/2203.11014 https://arxiv.org/abs/2203.11014

[45] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint Optimization of Tree-based Index and Deep Model for Recommender Systems. arXiv:cs.IR/1902.07565

[46] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning Tree-based Deep Model for Recommender Systems. https://doi.org/10.1145/3219819.3219826

[47] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning Optimal Tree Models Under Beam Search. arXiv:stat.ML/2006.15408

Algorithm 2 HSNN Serving

Input: MoNN Model, item-index mapping m
Output: Top k items

1: **for** each layer i in N **do**
2: Identify the index node for each item
3: Evaluate MoNN Model for f(user, index-node) using the features from the representative item in the index node.
4: Pass the MoNN model output along with previous MoNN model outputs to the ensemble layer to get the logit.
5: Select top-k nodes from this layer as candidates for next layer
6: **end for**
7: Publish top-k items to ranking stage.

A APPENDIX

A.1 MoNN Model Configurations

Besides FLOPs and theoretical cost for 4 MoNN models with different complexities shown in Table 1, Table 5 provides additional information on the model configurations such as the number of features and hyper-parameters of user, item and interaction towers.

A.2 HSNN Serving Algorithm

HSNN serving has two parts: the serving of the MoNN model and the inference against a hierarchical index.

A MoNN model is split into 4 parts for serving - the user tower, item tower, interaction tower and over-arch module.

- The user tower takes in user features and returns the user embedding. This happens at query time.
- The item tower takes in item features and returns the item embedding and item-to-index mapping. An index of items is built and maintained asynchronously, with new items being immediately indexed and old items being deleted from the index.
- The interaction tower and over-arch model compute the interaction embedding and the logit at query time in the Ad Retrieval system.

The HSNN serving algorithm is provided in Algorithm 2. The HSNN serving is layer-wise. For each layer, the representative index node of the item is selected and its features are used as input to the corresponding MoNN model. The MoNN model output is passed as input to the next layer MoNN model's linear ensemble layer to get the logit. Only the top-k nodes are selected for each layer.

Model	Num Features	Tower Params
Two Tower Tiny	user: O(100) item: O(100) interaction: 0	num_embed$_u$=1, dim$_u$=40 num_embed$_i$=1, dim$_i$=40
MoNN Small	user: O(100) item: O(100) interaction: O(10)	num_embed$_u$=8, dim$_u$=40 num_embed$_i$=1, dim$_i$=40 num_embed$_{ui}$=1, dim$_{ui}$=40
MoNN Medium	user: O(100) item: O(100) interaction: O(10)	num_embed$_u$=10, dim$_u$=48 num_embed$_i$=4, dim$_i$=48 num_embed$_{ui}$=1, dim$_{ui}$=48
MoNN Large	user: O(1000) item: O(100) interaction: O(100)	num_embed$_u$=30, dim$_u$=48 num_embed$_i$=8, dim$_i$=48 num_embed$_{ui}$=1, dim$_{ui}$=48
MoNN XL	user: O(1000) item: O(1000) interaction: O(100)	num_embed$_u$=80, dim$_u$=96 num_embed$_i$=4, dim$_i$=96 num_embed$_{ui}$=1, dim$_{ui}$=96

Table 5: MoNNs model complexities. User (u), Item (i) and Interaction (ux) are abbreviated for brevity.